

Project Report: DevSecOps Deployment of a Netflix-like Application on AWS

Project Title: End-to-End DevSecOps Pipeline for a Containerized Application on AWS

Student Name: Ammar Ahmad **Roll Number:** 22i-1267

Date: 30th March 2026

Project Demonstration

Video: https://drive.google.com/file/d/1uggmx_9TrYVdPmyQV0H54rmDYDXG0FSi/view?usp=drive_link

1. Executive Summary

This project aimed to design, implement, and secure a complete DevSecOps pipeline for deploying a Netflix-like application. The infrastructure was built on Amazon Web Services (AWS), utilizing a suite of modern tools including Docker, Jenkins, SonarQube, Trivy, Prometheus, Grafana, and Kubernetes (EKS). The primary goal was to automate the software delivery lifecycle, integrating security scanning at every stage ("Shift Left") and establishing robust monitoring for the application and infrastructure.

The project successfully established a CI/CD pipeline in Jenkins that automated code checkout, dependency analysis, static code analysis with SonarQube, and vulnerability scanning with Trivy. The application was containerized with Docker and pushed to Docker Hub. Monitoring was set up using Prometheus and Grafana to visualize metrics from the Jenkins server and the infrastructure. The final stage involved deploying the application to an Amazon EKS cluster, managed via ArgoCD. While the project was successfully completed, it was a complex endeavor that presented numerous challenges, requiring significant troubleshooting and iterative refinement.

2. Project Demonstration Video

A comprehensive demonstration of the entire project is available. The video showcases:

- EC2 instance setup and configuration
- Jenkins pipeline execution with all security stages
- SonarQube code quality analysis results
- Trivy vulnerability scanning outputs
- Prometheus targets and metrics collection
- Grafana dashboards visualizing system metrics
- Application deployment on EKS cluster
- ArgoCD synchronization and GitOps workflow

Project Video

Link: https://drive.google.com/file/d/1uggmx_9TrYVdPmyQV0H54rmDYDXG0FSi/view?usp=drive_link

Note: The video covers the complete end-to-end implementation, including troubleshooting steps and final verification of all components.

3. Project Objectives

- **Infrastructure Provisioning:** Launch and configure AWS EC2 instances to serve as the primary build/CI-CD server and the monitoring server.
 - **Application Containerization:** Dockerize the Netflix application, integrating the TMDB API key for dynamic content.
 - **CI/CD Pipeline Implementation:** Build an automated Jenkins pipeline incorporating code checkout, build, security scanning, and deployment steps.
 - **Security Integration ("Shift Left"):** Integrate SonarQube for code quality and security analysis, and Trivy for dependency and container image vulnerability scanning within the pipeline.
 - **Monitoring and Observability:** Deploy Prometheus and Grafana on a dedicated instance to monitor the health and performance of the Jenkins server, EC2 infrastructure, and the final Kubernetes cluster.
 - **Orchestration and Deployment:** Deploy the application to an Amazon EKS cluster, utilizing ArgoCD for GitOps-based continuous delivery.
-

4. Project Architecture & Tools Used

Component	Specification	Purpose
Jenkins/Build Server	AWS EC2 <code>m7i-flex.large</code>	Primary CI/CD server running Jenkins, Docker, SonarQube (containerized), and build tools
Monitoring Server	AWS EC2 <code>t3.small</code>	Dedicated instance for Prometheus and Grafana to avoid performance impact on the CI/CD pipeline
Containerization	Docker	Building and running the application container
CI/CD	Jenkins	Declarative pipeline for automation
Code Quality & Security	SonarQube, Trivy, OWASP Dependency-Check	Static analysis, vulnerability scanning
Artifact Registry	Docker Hub	Storing the final application image
Orchestration	Amazon EKS	Managed Kubernetes for production deployment
GitOps	ArgoCD	Deploying and syncing application state
Monitoring	Prometheus, Node Exporter, Grafana	Metrics collection and visualization
Source Control	GitHub	Version control for application code and Kubernetes manifests

The project architecture consisted of several key components distributed across two EC2 instances to optimize resource utilization within the AWS free tier constraints:

5. Implementation Phases and Challenges

The project was executed in distinct phases. Each phase presented its own set of challenges that were systematically resolved.

Phase 1: EC2 Setup and Application Initialization

- **Action:**
 - Two EC2 instances were launched: an `m7i-flex.large` (2 vCPUs, 8 GB RAM) for Jenkins and build operations, and a `t3.small` (2 vCPUs, 2 GB RAM) for monitoring tools.
 - Elastic IPs were associated with both instances for consistent access.
 - The application code was cloned from a GitHub repository.
 - Docker was installed on the Jenkins server, and a base container was built and run to verify the application.
- **Challenges & Resolution:**
 - The primary challenge was ensuring all necessary ports were open in the respective security groups (Jenkins: 8080, 9000; Monitoring: 9090, 3000).
 - A key step was obtaining and configuring the TMDb API key. Initially, the app failed to load movie data. This was resolved by rebuilding the Docker image using the `--build-arg` flag to pass the API key during the build process.
 - Resource consideration: The `t3.small` instance for monitoring was chosen to stay within free tier eligibility while ensuring adequate performance for Prometheus and Grafana.

Phase 2: CI/CD Server Setup (Jenkins)

- **Action:** Jenkins was installed and configured on the `m7i-flex.large` instance. Necessary plugins for Java, NodeJS, Docker, OWASP Dependency-Check, and SonarQube Scanner were installed. Docker Hub credentials were securely added to Jenkins' credential store.
- **Challenges & Resolution:**
 - The main difficulty was ensuring the correct versions of JDK (OpenJDK 17) and NodeJS were installed and globally configured in Jenkins.
 - Initial pipeline runs failed due to missing tools, which was fixed by correctly configuring the JDK and NodeJS installations under "Global Tool Configuration."
 - The `m7i-flex.large` instance provided ample CPU and memory for running Jenkins alongside containerized tools like SonarQube without performance degradation.

Phase 3: CI/CD Pipeline Creation & Security Integration

- **Action:** A declarative Jenkins pipeline (Jenkinsfile) was created with stages for:
 - Code Checkout
 - SonarQube Analysis (Static Code Analysis)
 - OWASP Dependency-Check (Dependency Scanning)
 - Trivy Filesystem Scan (Codebase Vulnerability Scan)
 - Docker Build & Push (to Docker Hub)
 - Trivy Image Scan (Container Image Vulnerability Scan)
 - Deployment to a test container
- **Challenges & Resolution:**
 - **SonarQube Integration:** Connecting Jenkins to the SonarQube server (running in its own container) was problematic. The issue was resolved by ensuring the SonarQube server was on the same network and correctly configuring the server URL and authentication token in Jenkins' system configuration.
 - **Trivy Scans:** The Trivy filesystem scan initially failed due to permission errors. This was resolved by ensuring the Jenkins user had the necessary permissions to access the workspace directory.
 - **Pipeline Syntax:** Crafting the correct pipeline syntax, especially for `withSonarQubeEnv` and `waitForQualityGate`, required multiple iterations and referencing official documentation.

Phase 4: Monitoring Setup (Prometheus & Grafana)

- **Action:**
 - Prometheus and Node Exporter were installed and configured as systemd services on the dedicated `t3.small` monitoring instance.
 - The `prometheus.yml` file was configured to scrape metrics from the Jenkins server (`m7i-flex.large` instance) on port 9090 and from Node Exporter on the monitoring instance itself.
 - Grafana was installed on the same `t3.small` instance, and Prometheus was added as a data source.
 - A pre-configured dashboard (ID 1860 for Node Exporter) was imported for visualization.
- **Challenges & Resolution:**
 - The main challenge was configuring the Jenkins Prometheus plugin and ensuring the metrics endpoint was accessible from the monitoring instance. This required

installing the "Prometheus Metrics" plugin in Jenkins and adding the correct `metrics_path` ('/prometheus') in the `prometheus.yml` file.

- Network security: Both instances needed appropriate security group rules to allow communication on ports 9090 (Prometheus) and 9100 (Node Exporter).
- The `t3.small` instance proved sufficient for running both Prometheus and Grafana simultaneously without resource contention.

Phase 5: Kubernetes Deployment (EKS)

- **Action:**

- The AWS CLI was installed and configured on the Jenkins server.
- An Amazon EKS cluster was created (assumed via console or CLI).
- The local `kubectl` context was updated to point to the new cluster.
- Prometheus Node Exporter was installed on the EKS cluster using Helm.
- ArgoCD was installed, and a new application was configured to deploy the application from its GitHub repository to the cluster, exposing it via a NodePort service.

- **Challenges & Resolution:**

- **EKS Access:** Configuring the IAM user and updating the kubeconfig file (`aws eks update-kubeconfig`) required careful attention to the correct cluster name and region.
- **Helm Installation:** The initial Helm repository URL was incorrect, leading to installation failures. This was resolved by using the correct URL `https://prometheus-community.github.io/helm-charts`.
- **ArgoCD Application Sync:** The ArgoCD application initially failed to sync due to a misconfigured path in the GitHub repository. The path was corrected to point to the directory containing the Kubernetes manifests.

6. Infrastructure Cost Optimization

To manage costs effectively within the AWS free tier constraints, the following decisions were made:

Component	Instance Type	Rationale
Jenkins/Build Server	m7i-flex.large	Chosen for better performance with Jenkins, Docker, and SonarQube; eligible under free tier for new accounts
Monitoring Server	t3.small	Dedicated instance to separate monitoring workloads; lower cost while still providing adequate resources for Prometheus and Grafana
Elastic IPs	Associated with both instances	Ensured consistent access without needing to update IPs after restarts
EKS Cluster	Managed service	Leveraged AWS-managed control plane to reduce operational overhead; paid only for worker nodes

7. Final Outcome & Verification

The project concluded with the successful operation of all components:

- **CI/CD Pipeline:** The Jenkins pipeline runs successfully on the m7i-flex.large instance, performing all security scans and pushing the Docker image to Docker Hub upon successful quality gate passes.
 - **Security Tools:** SonarQube reports code quality metrics, and Trivy identifies zero critical vulnerabilities in the final Docker image.
 - **Monitoring:** Prometheus on the t3.small instance scrapes metrics from both the Jenkins server and the EKS nodes. Grafana dashboards display real-time system metrics (CPU, memory) and Jenkins job performance data.
 - **Application Deployment:** The Netflix application is live and accessible on the EKS cluster via the public IP of a worker node on port 30007. The application correctly fetches and displays movie data using the integrated TMDB API key.
-

8. Conclusion

This project successfully demonstrated the implementation of a robust DevSecOps pipeline on AWS. By strategically selecting `m7i-flex.large` for compute-intensive CI/CD operations and `t3.small` for monitoring, the project maintained cost efficiency while delivering full functionality. By integrating security tools like SonarQube and Trivy directly into the CI/CD process, security vulnerabilities were identified and addressed early in the development lifecycle. The use of Prometheus and Grafana provided comprehensive observability into the infrastructure and application performance. Finally, deploying to a managed Kubernetes cluster with ArgoCD showcased a modern, scalable, and GitOps-driven approach to application delivery.

While the project was fraught with challenges—from configuration errors and syntax issues to integration complexities—each problem provided a valuable learning opportunity. Overcoming these obstacles reinforced critical skills in troubleshooting, cloud infrastructure, containerization, and the configuration of complex CI/CD pipelines. The final result is a fully automated, secure, and monitored deployment pipeline that can serve as a blueprint for modern application delivery.

9. References & Resources

- **Jenkins Official Documentation:** <https://www.jenkins.io/doc/>
- **SonarQube Documentation:** <https://docs.sonarqube.org/>
- **Trivy Documentation:** <https://aquasecurity.github.io/trivy/>
- **Prometheus Documentation:** <https://prometheus.io/docs/>
- **Grafana Documentation:** <https://grafana.com/docs/>
- **Amazon EKS Documentation:** <https://docs.aws.amazon.com/eks/>
- **ArgoCD Documentation:** <https://argo-cd.readthedocs.io/>
- **TMDB API:** <https://developers.themoviedb.org/3>
- **AWS EC2 Instance Types:** <https://aws.amazon.com/ec2/instance-types/>